# SA-HEACS: Server-Assisted Hybrid Execution and AI-Optimized Cloud Shielding Against Automated Reverse Engineering

Oct 15, 2026 · @Kai

## Abstract

SA-HEACS keeps a product's most valuable logic on servers the vendor controls, so client-side reverse engineering recovers an interface rather than the protected algorithm. Automated, AI-assisted reverse engineering pipelines now chain decompilers such as Ghidra, dynamic instrumentation such as Frida, and large language models to recover program semantics far faster than a human analyst. Classical client-side defenses (obfuscation, anti-debugging, tamper-proofing) raise cost but cannot prevent recovery in general, a limit made precise by Barak et al.'s impossibility result for virtual black-box obfuscation.

We present Server-Assisted Hybrid Execution and AI-Optimized Cloud Shielding (SA-HEACS), an architecture with three parts: a thin client shell that holds no high-value intellectual property (IP), attested RPC to segmented server clusters that execute that IP, and a learned orchestrator that decides execution locality, micro-batches requests, and prefetches likely work to offset the latency and cost of remote execution. We add a Dynamic Feature Promotion/Demotion Pipeline that moves each feature between cloud-only and client-local execution as its IP value and clone risk change.

We give a threat model, pseudocode and runnable Python for the orchestrator and lifecycle policy, and a discrete-event simulation of cost and latency. The analysis shows SA-HEACS removes the protected code from the attacker's machine, which defeats static decompilation and memory dumping of that code. It does not stop black-box functionality extraction through the API: query-based model extraction, output-based cloning, and account abuse remain, and we treat them as residual risks with rate-limiting, watermarking, and anomaly-detection mitigations rather than claiming them solved.

# 1. Introduction

Code that runs on hardware the attacker controls can eventually be read, so the strongest protection for an algorithm is to not ship it. Software protection has long accepted that every client-side defense is a cost multiplier, not a barrier [1, 2, 4]. Studies of professional attackers show that they defeat layered obfuscation by combining static and dynamic analysis and by focusing on the small region that holds the asset [11].

Two shifts make that cost multiplier smaller. First, analysis tools are scriptable end to end: Ghidra exposes a headless analyzer and scripting API [19], and Frida injects JavaScript hooks into running processes to trace, patch, and dump memory [18]. Second, learned models now lift binaries toward readable source and recover names and types: LLM4Decompile trains language models to decompile x86 binaries [13], and DIRTY recovers variable names and types from decompiler output [14]. An automated reverse engineering agent (the original SA-HEACS blueprint calls such a toolkit "REA") can therefore chain dumping, decompilation, and semantic recovery with little human effort. The scarce resource for the attacker is no longer analyst time; it is access to the code.

SA-HEACS removes that access for the code that matters. It splits an application into a thin client shell, which handles input, local rendering and commodity features, and a set of server-side services that hold the high-value IP. The idea of hiding a program slice on a trusted server is old [10], as is offloading work from a constrained client [7, 8, 9]. What is new here is the combination of three things: an explicit IP-centric partitioning rule, a learned orchestrator that makes the economics of remote execution workable through locality routing, micro-batching and prefetch, and a lifecycle policy that moves features between tiers as their value decays.

This paper makes five contributions:

1. An architecture that separates a zero-IP client shell from attested, segmented server clusters, with a request protocol (Section 4).
2. A threat model that states which reverse-engineering attacks the design removes and which it does not (Sections 3 and 7).
3. An orchestrator design with pseudocode and runnable Python for locality routing, micro-batching, and Markov prefetch (Section 5).
4. A Dynamic Feature Promotion/Demotion Pipeline that scores each feature's IP value and clone risk and moves it between cloud-only and client-local execution (Section 6).
5. An evaluation methodology and a reproducible simulation of cost and latency trade-offs, with the limits of what simulation can show (Section 8).

# 2. Background and related work

Prior work establishes that client-side protection only delays analysis, that server-side execution has been used for performance more than for secrecy, and that a remote API leaks its function through its outputs. SA-HEACS sits where those three lines meet.

## 2.1 Obfuscation and its limits

Collberg, Thomborson and Low catalogued obfuscating transformations by potency, resilience and cost [1], and later framed obfuscation, watermarking and tamper-proofing as complementary tools [2, 12]. Barak et al. proved that virtual black-box obfuscation is impossible for general programs: some functions leak properties through any obfuscated form [3]. Indistinguishability obfuscation is constructible in theory [5] but remains far too slow for production code. Empirically, Schrittwieser et al. found that most practical obfuscations fall to modern static and dynamic analysis given enough effort [4], and Banescu et al. showed that symbolic execution defeats many transformations unless they are designed against it [6]. Ceccato et al. observed professional penetration testers and found they locate and attack the protected asset rather than deobfuscating the whole program [11].

## 2.2 Automated and learning-based reverse engineering

Scriptable platforms turn reverse engineering into a pipeline. Ghidra provides headless batch analysis and a scripting API [19]; Frida injects instrumentation into live processes on desktop and mobile platforms [18]. Learned models then improve the output: DIRTY predicts variable names and types for decompiled code [14], and LLM4Decompile reports end-to-end decompilation of compiled binaries by fine-tuned language models [13]. These tools lower the human effort per target, which weakens defenses whose value rested on analyst time.

## 2.3 Program splitting and computation offloading

Zhang and Gupta proposed hiding critical program slices on a trusted server so the client binary is incomplete [10]; SA-HEACS generalizes this from slices to whole features with a managed lifecycle. Offloading systems such as MAUI [7], CloneCloud [8] and Neurosurgeon [9] partition work between device and cloud to save energy or latency, and cloud gaming systems stream rendered output to thin clients [32]. These systems optimize performance; secrecy of the offloaded code is not their goal, so they freely move code to the client when that is faster. SA-HEACS reverses the priority: security tier is a hard constraint and performance is optimized inside it.

### 2.4 Attestation

Remote attestation lets a server judge the state of a client before serving it. RFC 9334 defines the roles of attester, verifier and relying party [20]. Mobile platforms provide attestation services, Google Play Integrity [22] and Apple App Attest [23], and trusted execution environments such as Intel SGX add hardware-rooted quotes [21]. Attestation raises the cost of driving the API from a modified client, but it does not keep a legitimate client from observing its own inputs and outputs.

### 2.5 Model extraction through APIs

Keeping code on a server does not keep its function secret. Tramèr et al. extracted models from commercial prediction APIs through queries alone [15]; Orekondy et al. trained functionally similar "knockoff" networks from black-box outputs [30]; Jagielski et al. studied high-fidelity extraction [16]. Defenses include query-distribution monitoring such as PRADA [17] and watermarks embedded in a model's behavior [28]. These results bound what SA-HEACS can claim (Section 7).

### 2.6 Serving systems and batching

The orchestrator draws on ML serving work. Clipper introduced adaptive batching for latency-bounded prediction serving [24]; Orca added iteration-level scheduling for transformer inference [25]; vLLM's PagedAttention raised batch capacity by managing key-value cache memory [26]. Resource Central predicts VM behavior to inform allocation [29], and predictive prefetching has long been used to hide web latency [31]. Dean and Barroso describe why tail latency, not the mean, governs user experience at scale [27].

## 3. Threat model

The attacker fully controls the client device and holds a valid account, but cannot read server memory or storage. This is the standard man-at-the-end setting [12] with a trusted server added.

**Assets.** (A1) Proprietary algorithms and model weights in cloud-only features. (A2) Server-side state such as cached intermediate results. (A3) The service's availability and cost envelope. (A4) Lower-value code shipped in the client shell, which is explicitly not protected beyond standard obfuscation.

**Attacker capabilities.** The attacker can decompile and patch the client binary, hook any function with dynamic instrumentation, read and modify process memory, intercept traffic after TLS termination on their own device, and script all of this with an AI agent that plans, runs tools, and summarizes findings. They can create many accounts, subject to the vendor's sign-up controls, and send any sequence of valid queries.

**Trust assumptions.** The server fleet, its hypervisor, and the build pipeline are trusted; insiders and cloud-provider compromise are out of scope. Platform attestation services [22, 23] are assumed sound, though we treat a bypassed attestation as a degraded mode rather than a total failure. Side channels on shared server hardware are out of scope but noted in Section 9.

**Attacker goals.**

| Goal | Example | In scope |
|---|---|---|
| G1 Code recovery | Recover source or weights of a cloud-only feature | Yes |
| G2 Functional cloning | Train or build a substitute from input and output pairs | Yes, as residual risk |
| G3 Free-riding | Use a cloned or patched client to call the API without paying | Yes |
| G4 Shell cloning | Copy the client UI and commodity features | Accepted |
| G5 Denial of service | Exhaust GPU capacity with costly requests | Partly |
| G6 Server compromise | Exploit the RPC surface to read server memory | Out of scope; standard hardening |

## 4. SA-HEACS architecture

SA-HEACS has three components joined by one attested request path: the client shell, the orchestrator, and the secure clusters (Figure 1).

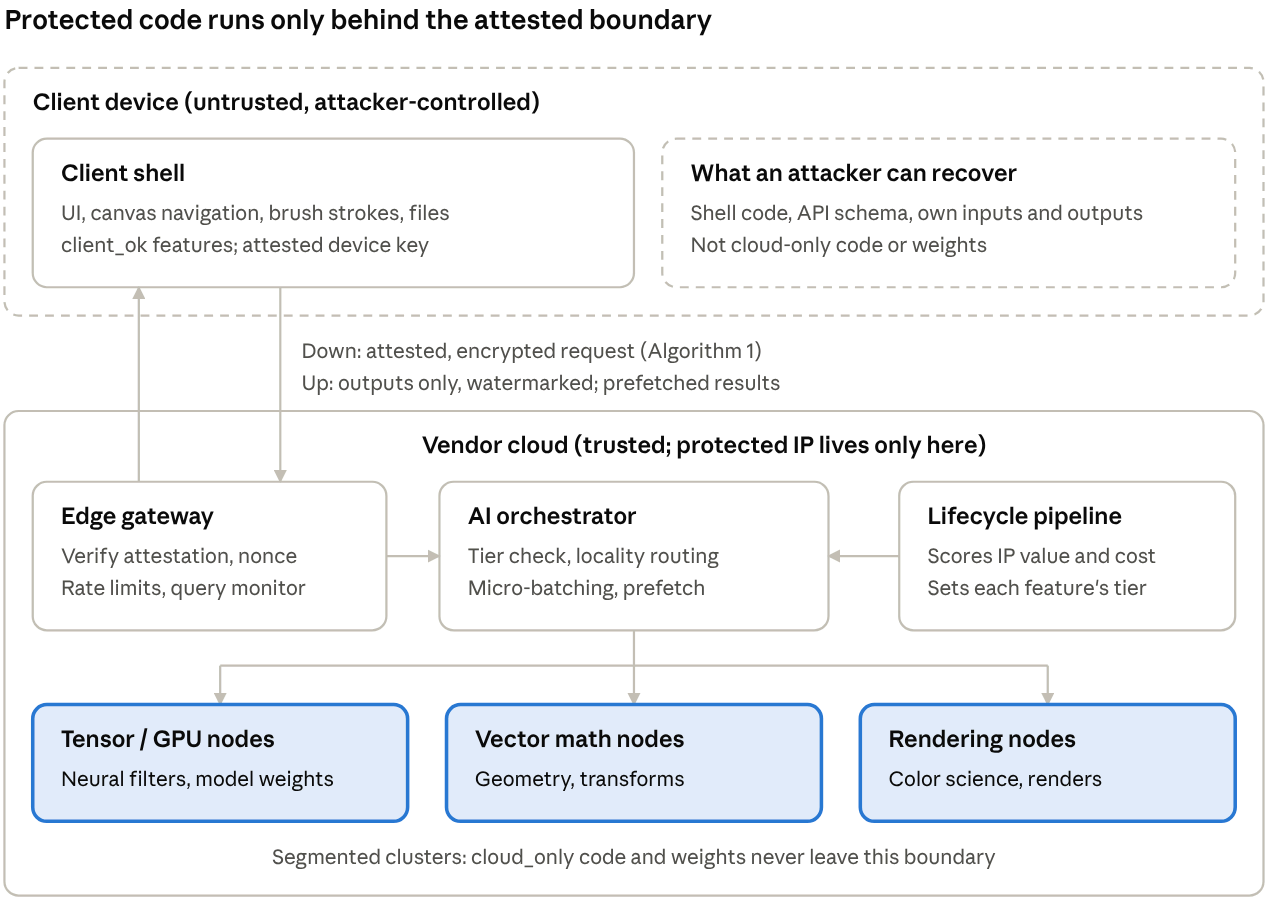



Figure 1 · SA-HEACS architecture: client shell, attested RPC, orchestrator, segmented clusters

Everything above the cloud boundary is assumed readable by the attacker; the blue clusters hold the protected IP and return only outputs.

## 4.1 Client shell (the untrusted boundary)

The shell handles input, canvas navigation, local brush strokes, file handling, UI rendering, and any feature in the `client_ok` tier. It holds no cloud-only code or weights. Its only secrets are a device identity bound to platform attestation and short-lived session keys. We assume it will be fully decompiled and cloned; the design goal is that a perfect clone gains a UI and commodity features, not the protected capabilities.

## 4.2 Secure clusters (the protected core)

Cloud-only features run as segmented microservices grouped by hardware need: tensor and GPU inference, vector and geometry math, and rendering or color-science pipelines. Each feature runs in its own service identity with least-privilege access to its weights.

Clients never receive code, weights, or intermediate tensors that would let a substitute be rebuilt more cheaply than by black-box queries; responses carry only the final output at the resolution the product needs.

## 4.3 Orchestrator (the control plane)

The orchestrator sits between the edge and the clusters. It enforces each feature's tier, routes `hybrid` and `client_ok` work to the cheaper side, groups compatible requests into micro-batches, and prefetches likely next operations. Section 5 details it.

## 4.4 Request protocol

Every remote call carries an attestation-bound signature, a fresh nonce, and an encrypted body. The server rejects stale or replayed requests and unattested devices before any protected code runs.

```
Algorithm 1  Attested invocation of a cloud-only feature
Client:
  1  eph ← X25519.generate()
  2  n ← random(16); t ← now()
  3  σ ← Sign(K_device, eph.pub ‖ n ‖ t)          // K_device bound to
platform attestation
  4  k ← HKDF(ECDH(eph, S_pub), salt = n)
  5  c ← AES-GCM.Enc(k, n[0:12], {feature, args}, aad = eph.pub ‖ n ‖ t)
  6  send (device_id, eph.pub, n, t, σ, c)
Server:
  7  if |now() − t| > Δ or n ∈ SeenNonces: reject
  8  if not Verify(AttestedKey[device_id], σ): reject
  9  if not RateLimiter.allow(account, feature): reject           // Section
7
 10  k ← HKDF(ECDH(S_priv, eph.pub), salt = n)
 11  req ← AES-GCM.Dec(k, n[0:12], c, aad)
 12  y ← Orchestrator.submit(req.feature, req.args)               // Section
5
 13  return AES-GCM.Enc(k, n[12:16] ‖ 0^8, Watermark(y), aad)
```

The runnable version, `attested_rpc.py` (Appendix A.1), implements lines 1 to 8 and 10 to 13, without the rate limiter and watermark, with the Python `cryptography` package and FastAPI. Running it shows a valid call returning the filtered result and a cloned client with an unattested key rejected with HTTP 401. In production, line 8 would consume a Play Integrity or App Attest verdict [22, 23] instead of a locally registered Ed25519 key.

# 5. The AI orchestrator

The orchestrator's job is to make remote execution cheap and fast enough that keeping IP server-side is commercially viable. It does three things, and the security tier constrains all of them: no optimization may move a `cloud_only` feature to the client.

## 5.1 Locality routing

For `hybrid` and `client_ok` features, a classifier trained on telemetry predicts whether remote execution will finish sooner than local execution, from payload size, estimated work, measured round-trip time, device capability, and battery. This follows the partitioning logic of MAUI and Neurosurgeon [7, 9], with the tier check placed first.

```
Algorithm 2  Route(r)
  1  if r.tier = cloud_only: return CLOUD                // hard security
constraint
  2  if r.tier = client_ok and r.rtt > RTT_MAX: return LOCAL
  3  p ← Model.P(remote faster | payload, work, rtt, device, battery)
  4  return CLOUD if p ≥ 0.5 else LOCAL
```

`orchestrator.py` (Appendix A.2) trains a scikit-learn `GradientBoostingClassifier` on 3,000 synthetic telemetry records and reaches 0.956 accuracy on 1,000 held-out records. The labels come from a simple latency model, so this number shows the pipeline works, not that a real router would be this accurate.

## 5.2 Micro-batching

Accelerators amortize fixed per-call cost (kernel launch, weight reads) across a batch. The orchestrator holds requests for the same feature for up to a window of W milliseconds or until B requests arrive, then runs one batched call, as Clipper does for model serving [24]. Requests from different tenants share a batch, but each row's output is returned only to its sender, and no state persists across rows.

```
Algorithm 3  MicroBatcher.submit(feature f, input x)
  1  Q[f].append((x, future))
  2  if |Q[f]| ≥ B: Flush(f)
  3  else if no timer for f: start timer(W, Flush(f))
  4  return await future
Flush(f):
  5  items ← Q[f]; Q[f] ← []
  6  Y ← Kernel_f(stack(x for x in items))            // one accelerator
call
  7  for each (item, y): item.future.resolve(y)
```

The asyncio implementation in Appendix A.2 serves a burst of 200 requests in 4 batched calls. Section 8 measures the cost and latency trade-off of W.

### 5.3 Predictive prefetch

A first-order Markov model over each user's feature sequence predicts the next operation. When the user invokes feature a, the orchestrator speculatively computes the top-k features b with P(b | a) above a threshold and pushes the results to the client cache. A hit replaces a network round trip with a local read; a miss wastes compute. With a toy session of ten steps, the model in Appendix A.2 learns that after `denoise` the user goes to `upscale` or `colorize`.

```
Algorithm 4  OnInvoke(user u, feature a)
  1  M[u.prev][a] += 1; u.prev ← a
  2  for b in top_k(M[a]) with M[a][b] / Σ M[a] ≥ p_min:
  3      if Budget.allow(u, b): schedule speculative Run(b, u.state) at low
priority
```

Prefetched results are outputs, not code, so prefetch does not weaken the zero-footprint rule. It does increase the volume of outputs a client receives, which matters for extraction risk (Section 7).

## 6. Dynamic feature promotion and demotion pipeline

A feature's tier is a business decision that changes over time: new IP starts cloud-only, and it moves toward the client only when its secrecy is worth less than the cost and latency of serving it remotely. Moves toward the client are one-way in practice, because shipped code must be assumed public, so the pipeline makes demotion (back to the cloud) automatic and promotion to the client gated by a human reviewer.

## 6.1 Tiers

- **cloud_only (incubation).** Code and weights exist only on servers. New or high-value features start here.
- **hybrid.** The server stays authoritative, but an obfuscated or reduced-quality client fallback is allowed for offline use or poor networks.
- **client_ok (commoditized).** The feature ships in the client package, protected only by standard obfuscation [1], and runs locally to save server cost and latency.

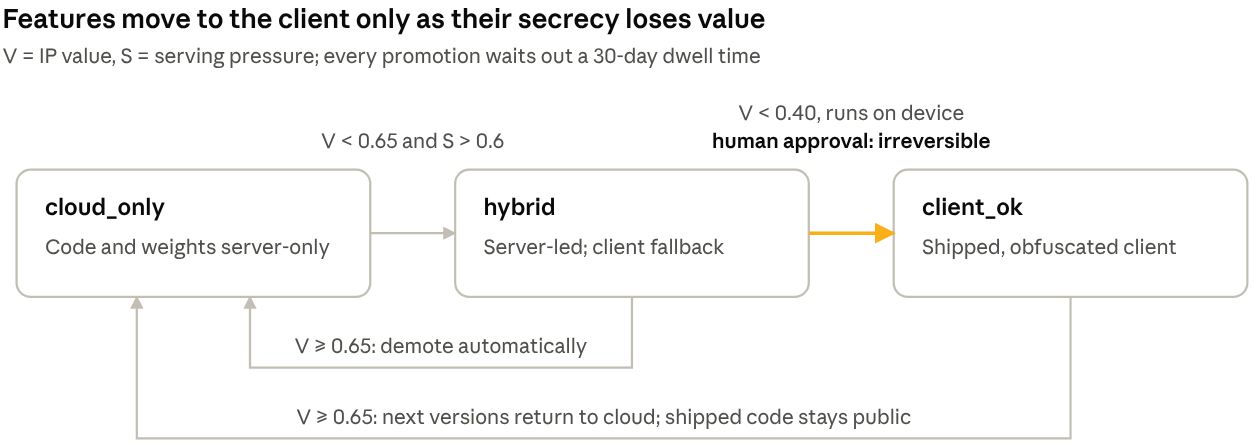



Figure 2 · feature tier state machine with the Section 6.3 thresholds

Only the move to client_ok needs a person to approve it, because shipped code cannot be recalled; every move back toward the cloud is automatic.

## 6.2 Scoring

The pipeline computes two scores per feature from product and telemetry signals. IP value V combines novelty N, share of premium revenue R, competitor parity C (how closely public competitors match the output), and whether the method is published, P:

```
V = 0.35\,N + 0.35\,R + 0.20\,(1 - C) + 0.10\,(1 - P)
```

Serving pressure S compares cost per 1,000 invocations c and p95 latency $\ell$ to budgets:

```
S = \min\!\left(1,\; \tfrac{1}{2}\,\tfrac{c}{c_{\max}} + \tfrac{1}
{2}\,\tfrac{\ell}{\ell_{\max}}\right)
```

The weights are illustrative; an organization would set them with its product and legal teams. Clone risk enters through C and P: once competitors reach parity or the method is published, secrecy protects little.

### 6.3 Transition policy

The policy uses a hysteresis band (V_hi = 0.65, V_lo = 0.40) and a minimum dwell time of 30 days so that noisy signals do not cause a feature to oscillate between tiers.

```
Algorithm 5  Decide(feature f, signals s)
  1  V ← IPValue(s); S ← ServingPressure(s)
  2  if V ≥ V_hi and f.tier ≠ cloud_only: return cloud_only        // demote
now; reversible
  3  if f.days_in_tier < MIN_DWELL: return f.tier
  4  if f.tier = cloud_only and V < V_hi and S > 0.6: return hybrid
  5  if f.tier = hybrid and V < V_lo and s.on_device_feasible:
  6      return client_ok   pending Approve(f)                    //
irreversible: human gate
  7  return f.tier
```

`lifecycle.py` (Appendix A.3) runs one simulated year for a feature whose novelty and revenue share decay while competitor parity rises and the method is published on day 201. The feature moves from cloud_only to hybrid on day 97 (V = 0.648, S = 0.908) and to client_ok on day 201 (V = 0.36), the day publication removes its remaining secrecy value.

### 6.4 Automated packaging

A promotion to client_ok triggers a build step that compiles the feature for client targets, applies the shell's standard obfuscation profile, and adds it to the next client release behind a feature flag. Promotion should never ship the server's exact weights or code if a distilled or reduced version is enough, since the server version may later become valuable again or share components with newer cloud-only features.

## 7. Security analysis

SA-HEACS removes code recovery (G1) for cloud-only features, because the code is not on the device; it reduces but does not remove functional cloning (G2) and free-riding (G3). The original blueprint rated every row "fully mitigated"; the table below separates what the architecture eliminates from what it only makes harder.

| Threat | Client-only protection | SA-HEACS, cloud-only features | Why |
|---|---|---|---|
| Static decompilation (Ghidra, IDA) | Delayed by obfuscation [1, 4] | Eliminated for the protected code | Code is not in the client package |
| Memory dumping and hooking (Frida) | Delayed by anti-debug and integrity checks | Eliminated for the protected code; client inputs and outputs remain visible | Computation runs server-side |
| AI-assisted decompilation and summarization | Increasingly effective [13, 14] | Recovers only the shell and the API schema | Nothing protected to lift |
| Black-box functional cloning via the API | Possible | Remains possible; slowed by rate limits, monitoring, output shaping | Outputs leak function [15, 16, 30] |
| Patched or cloned client driving the API | Possible | Raised cost: attestation, per-account quotas | Attestation can be bypassed on rooted or emulated devices |
| Shell and UI cloning | Possible | Accepted | Clone has no protected capability |
| Cost exhaustion (DoS on GPUs) | Not applicable | New risk | Remote compute is costly per request |

## 7.1 What is eliminated

Against G1, the argument is simple and does not depend on obfuscation strength: an analyst cannot recover an artifact that never reaches their machine. Barak et al.'s impossibility result [3] limits what any shipped program can hide; SA-HEACS avoids that limit by not shipping the program. This holds only if the server's responses do not contain the code or weights in another form, so responses must not include debugging traces, intermediate tensors, or model files.

### 7.2 What remains: extraction through the API

A server-side function is still an oracle. Tramèr et al. [15] and Orekondy et al. [30] show that query access alone can produce a useful substitute model, and an AI agent can automate the query campaign as easily as it automates decompilation. SA-HEACS therefore treats G2 as a residual risk with layered controls:

- **Per-account and per-device quotas** tied to attested identities, so that query volume costs real accounts and devices.
- **Query-distribution monitoring** in the style of PRADA [17], flagging accounts whose inputs look synthetic or adversarially spaced.
- **Output shaping**: return the product-level result (for example an 8-bit image) rather than logits, confidences, or high-precision intermediates that speed up extraction [15].
- **Watermarking** of model behavior or outputs [28], so that a stolen substitute can be identified even if it cannot be prevented.

These controls raise the price of a clone; none makes it impossible. The honest claim is that SA-HEACS converts a white-box attack into a black-box one and makes the black-box attack measurable and rate-limited.

### 7.3 Free-riding and attestation

Attestation [20, 22, 23] lets the server refuse requests from modified clients, but attackers can bypass it on compromised devices, and a legitimate unmodified client can still be scripted through accessibility or UI automation. Attestation is therefore one signal for the rate limiter and anomaly detector, not a gate that is assumed unbreakable.

### 7.4 New attack surface

Moving IP to the server creates risks client-only software does not have: cost-exhaustion attacks against GPU capacity, injection or deserialization bugs in the RPC layer, and cross-tenant leakage if batching or caching mixes state between users. Micro-batching must keep per-row isolation, and prefetch caches must be keyed per user.

## 8. Evaluation methodology and simulated results

In a simulation with stated assumptions, micro-batching makes server-side execution roughly 8 times cheaper at a cost of a few milliseconds, and Markov prefetch hides most network latency but multiplies compute. These numbers come from a model, not a deployment; Section 8.3 lists what a real evaluation must measure.

## 8.1 Setup

`simulate.py` (Appendix A.4) uses SimPy for a discrete-event model of one protected feature. Requests arrive as a Poisson process at 2,000 per second. A batched kernel call costs 4.0 ms fixed plus 0.15 ms per item, with at most 64 items per batch. The GPU pool is sized to keep utilization at or below 70%. Network round-trip time is 40 ms and a prefetched local hit costs 5 ms. Each configuration simulates 20 seconds, about 40,000 requests.

## 8.2 Results

**Batching window.** Figure 3 plots GPU time per 1,000 requests against server p95 latency for windows from 0 to 32 ms. Without batching, each request pays the full fixed cost, so 1,000 requests take 4,150 GPU-ms on a pool of 12 GPUs. A 5 ms window cuts this to 513 GPU-ms on 2 GPUs while server p95 rises from 5.0 ms to 11.0 ms. Past 20 ms, batches hit the 64-item cap and cost stops falling while latency keeps rising.

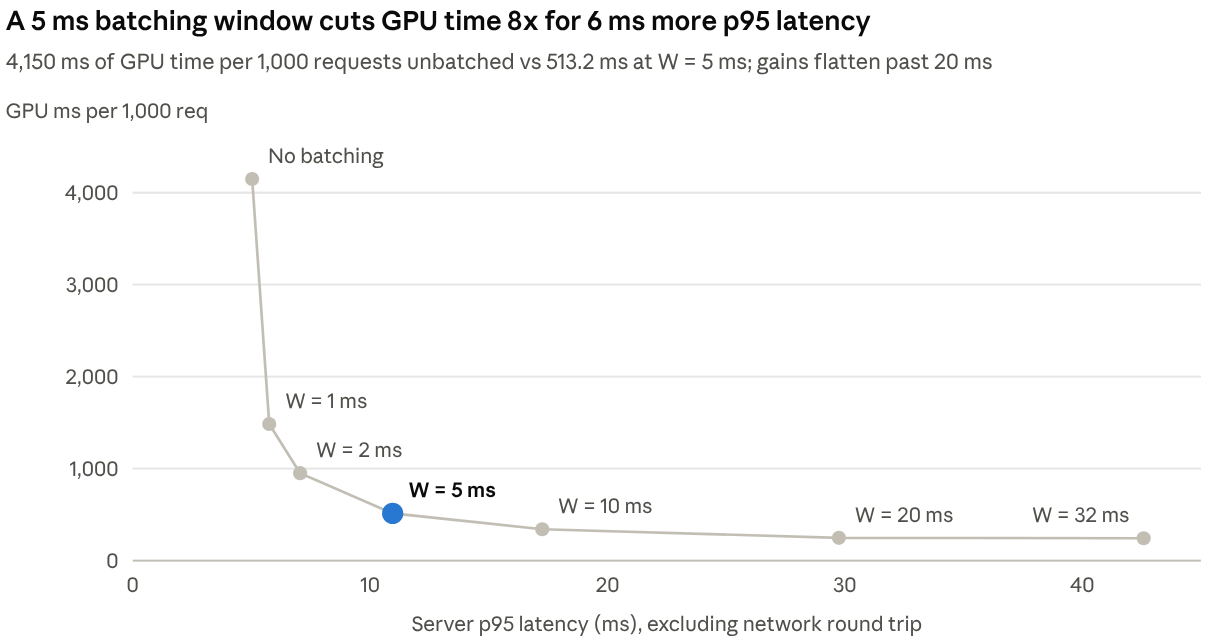



Figure 3 · simulate.py, SimPy model, 2,000 req/s, assumed kernel costs (not measured)

The knee of this curve depends on traffic: a feature with 20 requests per second will rarely fill a batch within 5 ms, so low-traffic features gain far less. This supports the lifecycle policy in Section 6: features that cannot batch well carry higher serving pressure S and become candidates for promotion sooner.

**Prefetch depth.** Figure 4 varies the number of speculated next features k for synthetic sessions drawn from a skewed Markov chain over 12 features. With k = 2, 68% of next steps are already in the client cache and mean perceived latency falls from 44.1 ms to 17.7 ms, but speculative work adds 132% to demand compute. The batching savings in Figure 3 are large enough to pay for k = 1 or 2; k = 4 adds 312% for a further 8 ms.

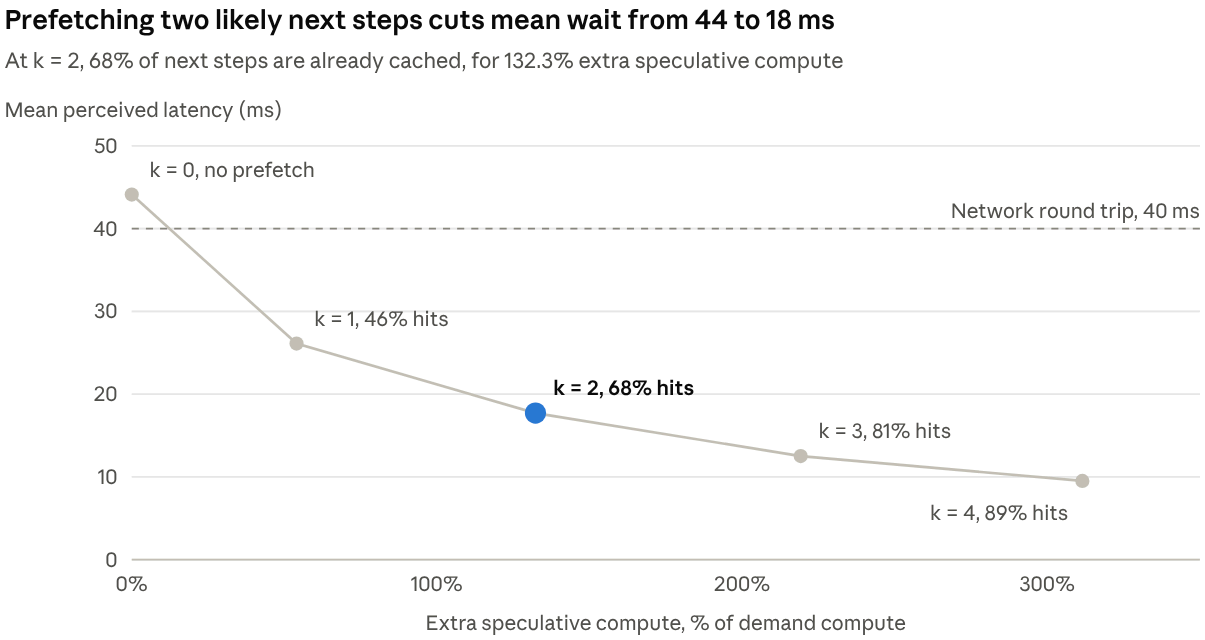



Figure 4 · simulate.py, 3,000 synthetic sessions over 12 features, first third used for training

### 8.3 What a real evaluation must measure

The simulation fixes kernel costs, traffic and user behavior by assumption. A deployment study should report: (1) per-feature p50, p95 and p99 end-to-end latency across real networks, since tail latency dominates experience [27]; (2) GPU cost per 1,000 invocations with real traffic mixes and multi-tenant batching; (3) prefetch hit rate and wasted compute from real session logs; (4) a red-team exercise in which an AI-assisted reverse engineering agent targets the client shell, reporting what it recovers; and (5) an extraction study measuring how many queries a substitute model needs to reach a given fidelity, with and without the controls of Section 7.2 [15, 17, 30].

## 9. Limitations and discussion

SA-HEACS trades a code-secrecy problem for an operations problem, and several limits remain.

- **Black-box extraction.** The architecture cannot stop a patient attacker from cloning a function through its outputs (Section 7.2). Features whose value is easy to learn from input and output pairs gain less from server-side execution than features with large hidden state or data.
- **Latency floor.** Interactive operations below about one network round trip cannot move to the server without prediction. Prefetch helps only when user behavior is predictable, and the 40 ms round trip assumed here is optimistic for mobile networks.
- **Offline use.** Cloud-only features fail without connectivity. The original blueprint's answer, locking or queueing premium features offline, is a product cost that users will notice.
- **Cost.** Remote execution moves compute from users' devices to the vendor's bill. Batching helps only for features with enough concurrent traffic (Section 8.2).
- **Privacy and compliance.** User content now leaves the device. Regulated data, data residency rules and user trust may forbid this for some customers, and any server-side cache becomes a breach target.
- **Shared-hardware side channels.** Multi-tenant GPUs and CPUs can leak information across tenants. We scope this out, but a production system needs isolation between batches of different customers where the threat warrants it.
- **Simulation only.** Section 8 uses assumed costs and synthetic sessions. The numbers illustrate trade-offs, not achievable performance.
- **Learned control plane.** The router and prefetcher are themselves models that can drift or be gamed, for example by a user who crafts sequences to trigger costly speculation. The tier constraint is enforced in code, not learned, so a misbehaving model can waste money but cannot leak a cloud-only feature.

The lifecycle pipeline also changes how teams think about protection. Instead of asking whether a feature can be protected on the client, they ask how long its secrecy is worth the serving cost. That question has a business answer that changes over time, which is what the promotion and demotion policy encodes.

## 10. Conclusion

Client-side protection cannot win against an attacker who owns the machine, and AI-assisted tooling shortens the time it takes to lose. SA-HEACS accepts this and keeps the code that matters off the machine: a zero-IP client shell, attested and encrypted calls to

segmented server clusters, and an orchestrator that uses locality routing, micro-batching and prefetch to keep remote execution affordable. A promotion and demotion pipeline moves each feature to the client once its secrecy no longer pays for its serving cost.

The design eliminates static and dynamic recovery of cloud-only code. It does not eliminate functional cloning through the API, which becomes the main residual risk and calls for rate limiting, query monitoring, output shaping and watermarking. Future work should evaluate the design on a real application with production traffic, run an AI-assisted red team against the client shell, and measure extraction cost against the deployed controls.

## References


1. C. Collberg, C. Thomborson, D. Low. *A Taxonomy of Obfuscating Transformations.* Technical Report 148, Department of Computer Science, University of Auckland, 1997.
2. C. Collberg, C. Thomborson. Watermarking, tamper-proofing, and obfuscation: tools for software protection. *IEEE Transactions on Software Engineering* 28(8):735–746, 2002. doi:10.1109/TSE.2002.1027797
3. B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. Vadhan, K. Yang. On the (im)possibility of obfuscating programs. *Journal of the ACM* 59(2), 2012 (CRYPTO 2001). doi:10.1145/2160158.2160159
4. S. Schrittwieser, S. Katzenbeisser, J. Kinder, G. Merzdovnik, E. Weippl. Protecting software through obfuscation: can it keep pace with progress in code analysis? *ACM Computing Surveys* 49(1), 2016. eprint
5. S. Garg, C. Gentry, S. Halevi, M. Raykova, A. Sahai, B. Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. *FOCS*, 2013. IACR ePrint 2013/451
6. S. Banescu, C. Collberg, V. Ganesh, Z. Newsham, A. Pretschner. Code obfuscation against symbolic execution attacks. *ACSAC*, 2016.
7. E. Cuervo, A. Balasubramanian, D. Cho, A. Wolman, S. Saroiu, R. Chandra, P. Bahl. MAUI: making smartphones last longer with code offload. *MobiSys*, 2010.
8. B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, A. Patti. CloneCloud: elastic execution between mobile device and cloud. *EuroSys*, 2011.
9. Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang. Neurosurgeon: collaborative intelligence between the cloud and mobile edge. *ASPLOS*, 2017.
10. X. Zhang, R. Gupta. Hiding program slices for software security. *CGO*, 2003. CERIAS record

11. M. Ceccato, P. Tonella, C. Basile, B. Coppens, B. De Sutter, P. Falcarin, M. Torchiano. How professional hackers understand protected code while performing attack tasks. *ICPC*, 2017. arXiv:1704.02774
12. C. Collberg, J. Nagra. *Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection.* Addison-Wesley, 2009.
13. H. Tan, Q. Luo, J. Li, Y. Zhang. LLM4Decompile: decompiling binary code with large language models. *EMNLP*, 2024. ACL Anthology
14. Q. Chen, J. Lacomis, E. J. Schwartz, C. Le Goues, G. Neubig, B. Vasilescu. Augmenting decompiler output with learned variable names and types. *USENIX Security*, 2022. arXiv:2108.06363
15. F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, T. Ristenpart. Stealing machine learning models via prediction APIs. *USENIX Security*, 2016. arXiv:1609.02943
16. M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, N. Papernot. High accuracy and high fidelity extraction of neural networks. *USENIX Security*, 2020. arXiv:1909.01838
17. M. Juuti, S. Szyller, S. Marchal, N. Asokan. PRADA: protecting against DNN model stealing attacks. *IEEE EuroS&P*, 2019. arXiv:1805.02628
18. Frida: dynamic instrumentation toolkit. Documentation. frida.re
19. National Security Agency. Ghidra software reverse engineering framework. GitHub
20. H. Birkholz, D. Thaler, M. Richardson, N. Smith, W. Pan. Remote ATtestation procedureS (RATS) Architecture. RFC 9334, IETF, 2023. rfc-editor.org
21. V. Costan, S. Devadas. Intel SGX explained. IACR ePrint 2016/086. eprint.iacr.org
22. Google. Play Integrity API. Android Developers documentation. developer.android.com
23. Apple. Establishing your app's integrity (App Attest). Apple Developer documentation. developer.apple.com
24. D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, I. Stoica. Clipper: a low-latency online prediction serving system. *NSDI*, 2017. arXiv:1612.03079
25. G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun. Orca: a distributed serving system for transformer-based generative models. *OSDI*, 2022.
26. W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023. arXiv:2309.06180
27. J. Dean, L. A. Barroso. The tail at scale. *Communications of the ACM* 56(2):74–80, 2013. doi:10.1145/2408776.2408794

28. Y. Adi, C. Baum, M. Cisse, B. Pinkas, J. Keshet. Turning your weakness into a strength: watermarking deep neural networks by backdooring. *USENIX Security*, 2018. arXiv:1802.04633
29. E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, R. Bianchini. Resource Central: understanding and predicting workloads for improved resource management in large cloud platforms. *SOSP*, 2017.
30. T. Orekondy, B. Schiele, M. Fritz. Knockoff nets: stealing functionality of black-box models. *CVPR*, 2019. arXiv:1812.02766
31. V. N. Padmanabhan, J. C. Mogul. Using predictive prefetching to improve World Wide Web latency. *ACM SIGCOMM Computer Communication Review* 26(3), 1996.
32. C.-Y. Huang, C.-H. Hsu, Y.-C. Chang, K.-T. Chen. GamingAnywhere: an open cloud gaming system. *ACM MMSys*, 2013. project page
33. F. Pedregosa et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12:2825–2830, 2011.

Software used in the examples: pyca/cryptography, FastAPI, scikit-learn [33], SimPy, NumPy.

# Appendix A. Example code

All four programs run with Python 3.13 after `pip install cryptography fastapi httpx numpy scikit-learn simpy`; the outputs quoted in the paper come from these runs. They are teaching implementations: keys live in memory, telemetry is synthetic, and production concerns such as key rotation, persistence and observability are left out.

## A.1 attested_rpc.py: attested, encrypted invocation (Algorithm 1)

```
import base64, json, os, time
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric.ed25519 import
Ed25519PrivateKey, Ed25519PublicKey
from cryptography.hazmat.primitives.asymmetric.x25519 import
X25519PrivateKey, X25519PublicKey
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

b64 = lambda b: base64.b64encode(b).decode()
```

```
unb64 = base64.b64decode
raw = dict(encoding=serialization.Encoding.Raw,
format=serialization.PublicFormat.Raw)

def derive_key(shared, nonce):
    return HKDF(algorithm=hashes.SHA256(), length=32, salt=nonce, info=b"sa-
heacs/v1").derive(shared)

# ---- server ------------------------------------------------------------
SERVER_KEY = X25519PrivateKey.generate()
TRUSTED_DEVICE_KEYS = {}   # stands in for a Play Integrity / App Attest
verifier
SEEN_NONCES, MAX_SKEW_S = set(), 30

def proprietary_filter(pixels):
    """The protected IP. It exists only in the server process."""
    mean = sum(pixels) / len(pixels)
    return [round(0.8 * p + 0.2 * mean, 4) for p in pixels]

FEATURES = {"neural_filter_v3": proprietary_filter}

class Envelope(BaseModel):
    device_id: str; client_pub: str; nonce: str; ts: int; signature: str;
ciphertext: str

app = FastAPI(title="SA-HEACS secure cluster")

@app.post("/v1/invoke")
def invoke(env: Envelope):
    nonce = unb64(env.nonce)
    if abs(time.time() - env.ts) > MAX_SKEW_S or nonce in SEEN_NONCES:
        raise HTTPException(401, "stale or replayed request")
    device = TRUSTED_DEVICE_KEYS.get(env.device_id)
    if device is None:
        raise HTTPException(401, "device not attested")
    signed = unb64(env.client_pub) + nonce + str(env.ts).encode()
    try:
        device.verify(unb64(env.signature), signed)
    except Exception:
        raise HTTPException(401, "bad attestation signature")
    SEEN_NONCES.add(nonce)
    key =
derive_key(SERVER_KEY.exchange(X25519PublicKey.from_public_bytes(unb64(env.cl
```

```
ient_pub))), nonce)
    aes = AESGCM(key)
    req = json.loads(aes.decrypt(nonce[:12], unb64(env.ciphertext), signed))
    fn = FEATURES.get(req["feature"])
    if fn is None:
        raise HTTPException(404, "unknown feature")
    out = json.dumps({"result": fn(req["args"])}).encode()
    return {"ciphertext": b64(aes.encrypt(nonce[12:] + b"\x00" * 8, out,
signed))}

# ---- client shell: no proprietary logic, only identity and transport ------
class ClientShell:
    def __init__(self, http, server_pub, device_id, device_key):
        self.http, self.server_pub, self.device_id, self.device_key = http,
server_pub, device_id, device_key

    def call(self, feature, args):
        eph = X25519PrivateKey.generate()
        nonce, ts = os.urandom(16), int(time.time())
        pub = eph.public_key().public_bytes(**raw)
        signed = pub + nonce + str(ts).encode()
        aes = AESGCM(derive_key(eph.exchange(self.server_pub), nonce))
        body = json.dumps({"feature": feature, "args": args}).encode()
        env = Envelope(device_id=self.device_id, client_pub=b64(pub),
nonce=b64(nonce), ts=ts,
                       signature=b64(self.device_key.sign(signed)),
                       ciphertext=b64(aes.encrypt(nonce[:12], body, signed)))
        r = self.http.post("/v1/invoke", json=env.model_dump())
        r.raise_for_status()
        out = aes.decrypt(nonce[12:] + b"\x00" * 8, unb64(r.json()
["ciphertext"]), signed)
        return json.loads(out)["result"]

if __name__ == "__main__":
    from fastapi.testclient import TestClient
    device_key = Ed25519PrivateKey.generate()
    TRUSTED_DEVICE_KEYS["device-42"] = device_key.public_key()   # result of
a passed attestation
    shell = ClientShell(TestClient(app), SERVER_KEY.public_key(), "device-
42", device_key)
    print("server result:", shell.call("neural_filter_v3", [0.1, 0.5, 0.9]))
    rogue = ClientShell(TestClient(app), SERVER_KEY.public_key(), "device-
42", Ed25519PrivateKey.generate())
```

```
    try:
        rogue.call("neural_filter_v3", [0.1])
    except Exception as e:
        print("unattested clone rejected:", e.response.status_code,
e.response.json()["detail"])
```

Output:

```
server result: [0.18, 0.5, 0.82]
unattested clone rejected: 401 bad attestation signature
```

## A.2 orchestrator.py: routing, micro-batching, prefetch (Algorithms 2 to 4)

```
import asyncio, time
from collections import Counter, defaultdict
from dataclasses import dataclass
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier

@dataclass
class Request:
    feature: str
    tier: str              # "cloud_only" | "hybrid" | "client_ok" (set by
the lifecycle pipeline)
    payload_kb: float
    est_flops_g: float
    rtt_ms: float
    device_gflops: float
    battery: float

def features(r):
    return [r.payload_kb, r.est_flops_g, r.rtt_ms, r.device_gflops,
r.battery]

class LocalityRouter:
    """Security tier is a hard constraint; the model only optimizes inside
it."""
    def __init__(self):
        self.model = GradientBoostingClassifier(n_estimators=100,
max_depth=3)
    def fit(self, X, y):
        self.model.fit(X, y); return self
```

```
    def route(self, r):
        if r.tier == "cloud_only":
            return "cloud"
        if r.tier == "client_ok" and r.rtt_ms > 250:
            return "local"
        return "cloud" if self.model.predict_proba([features(r)])[0, 1] >=
0.5 else "local"

def synthetic_telemetry(n=4000, seed=0):
    rng = np.random.default_rng(seed)
    payload = rng.lognormal(4, 1, n); flops = rng.lognormal(2, 1.2, n)
    rtt = rng.gamma(2, 30, n); dev = rng.uniform(5, 200, n); batt =
rng.uniform(0, 1, n)
    local_ms = flops / dev * 1000
    remote_ms = rtt + payload * 8 / 50_000 * 1000 + flops / 2000 * 1000
    return np.column_stack([payload, flops, rtt, dev, batt]), (remote_ms <
local_ms).astype(int)

class MicroBatcher:
    def __init__(self, batch_fn, window_ms=8.0, max_batch=32):
        self.batch_fn, self.window, self.max_batch = batch_fn, window_ms /
1000, max_batch
        self.queues, self.timers, self.batches_run = defaultdict(list), {}, 0
    async def submit(self, feature, x):
        fut = asyncio.get_running_loop().create_future()
        q = self.queues[feature]; q.append((x, fut))
        if len(q) >= self.max_batch:
            self._flush(feature)
        elif feature not in self.timers:
            self.timers[feature] =
asyncio.get_running_loop().call_later(self.window, self._flush, feature)
        return await fut
    def _flush(self, feature):
        if (t := self.timers.pop(feature, None)) is not None:
            t.cancel()
        items, self.queues[feature] = self.queues[feature], []
        if not items:
            return
        out = self.batch_fn(feature, np.stack([x for x, _ in items]))   # one
kernel launch
        self.batches_run += 1
        for (_, fut), y in zip(items, out):
            fut.set_result(y)
```

```
class MarkovPrefetcher:
    def __init__(self, k=2, min_p=0.25):
        self.k, self.min_p, self.counts = k, min_p, defaultdict(Counter)
    def observe(self, prev, nxt):
        self.counts[prev][nxt] += 1
    def predict(self, current):
        c = self.counts[current]; total = sum(c.values()) or 1
        return [f for f, n in c.most_common(self.k) if n / total >=
self.min_p]

if __name__ == "__main__":
    X, y = synthetic_telemetry()
    router = LocalityRouter().fit(X[:3000], y[:3000])
    print(f"router held-out accuracy: {router.model.score(X[3000:],
y[3000:]):.3f}")
    print("cloud_only feature routes to:",
          router.route(Request("neural_filter_v3", "cloud_only", 20, 0.1,
400, 200, 1.0)))
    W = np.random.default_rng(1).normal(size=(16, 16))           # stand-in
for protected weights
    batcher = MicroBatcher(lambda f, xb: np.tanh(xb @ W), window_ms=5,
max_batch=64)
    async def burst():
        t = time.perf_counter()
        xs = np.random.default_rng(2).normal(size=(200, 16))
        await asyncio.gather(*(batcher.submit("neural_filter_v3", x) for x in
xs))
        return time.perf_counter() - t
    secs = asyncio.run(burst())
    print(f"200 requests served in {batcher.batches_run} batched calls
({secs*1000:.1f} ms)")
    pf = MarkovPrefetcher()
    session = ["open", "denoise", "upscale", "export", "open", "denoise",
"upscale", "open", "denoise", "colorize"]
    for a, b in zip(session, session[1:]):
        pf.observe(a, b)
    print("after 'denoise', prefetch:", pf.predict("denoise"))
```

Output:

```
router held-out accuracy: 0.956
cloud_only feature routes to: cloud
200 requests served in 4 batched calls (10.4 ms)
after 'denoise', prefetch: ['upscale', 'colorize']
```

## A.3 lifecycle.py: promotion and demotion (Algorithm 5)

```
from dataclasses import dataclass, field
from enum import Enum

class Tier(Enum):
    CLOUD_ONLY = "cloud_only"; HYBRID = "hybrid"; CLIENT_OK = "client_ok"

@dataclass
class FeatureSignals:
    novelty: float; revenue_share: float; competitor_parity: float;
published: bool
    cost_per_1k_usd: float; p95_latency_ms: float; on_device_feasible: bool

WEIGHTS = dict(novelty=0.35, revenue=0.35, parity=0.20, published=0.10)

def ip_value(s):
    return round(WEIGHTS["novelty"] * s.novelty + WEIGHTS["revenue"] *
s.revenue_share
                 + WEIGHTS["parity"] * (1 - s.competitor_parity)
                 + WEIGHTS["published"] * (0 if s.published else 1), 3)

def serving_pressure(s, cost_budget=2.0, latency_budget=120.0):
    return round(min(1.0, 0.5 * s.cost_per_1k_usd / cost_budget + 0.5 *
s.p95_latency_ms / latency_budget), 3)

@dataclass
class Feature:
    name: str
    tier: Tier = Tier.CLOUD_ONLY
    days_in_tier: int = 0
    history: list = field(default_factory=list)

PROTECT_HI, PROTECT_LO, MIN_DWELL_DAYS = 0.65, 0.40, 30

def decide(f, s):
    v, p = ip_value(s), serving_pressure(s)
```

```
    if v >= PROTECT_HI and f.tier != Tier.CLOUD_ONLY:
        return Tier.CLOUD_ONLY, f"demote: ip_value {v} >= {PROTECT_HI}"
    if f.days_in_tier < MIN_DWELL_DAYS:
        return f.tier, "hold: dwell time"
    if f.tier == Tier.CLOUD_ONLY and v < PROTECT_HI and p > 0.6:
        return Tier.HYBRID, f"promote: ip_value {v}, pressure {p}"
    if f.tier == Tier.HYBRID and v < PROTECT_LO and s.on_device_feasible:
        return Tier.CLIENT_OK, f"promote (needs approval): ip_value {v} <
{PROTECT_LO}"
    return f.tier, "hold"

def step(f, s, approve):
    target, why = decide(f, s)
    if target == Tier.CLIENT_OK and f.tier != Tier.CLIENT_OK and not
approve(f, why):
        target, why = f.tier, why + " -> rejected by reviewer"
    if target != f.tier:
        f.history.append((f.tier.value, target.value, why)); f.tier,
f.days_in_tier = target, 0
    else:
        f.days_in_tier += 1
    return f

if __name__ == "__main__":
    f = Feature("neural_filter_v3")
    for day in range(365):
        t = day / 365
        s = FeatureSignals(novelty=0.95 - 0.8 * t, revenue_share=0.6 - 0.45 *
t,
                           competitor_parity=min(1, 0.1 + 1.1 * t),
published=day > 200,
                           cost_per_1k_usd=1.8, p95_latency_ms=110,
on_device_feasible=day > 150)
        before = len(f.history)
        step(f, s, approve=lambda feat, why: True)
        if len(f.history) > before:
            old, new, why = f.history[-1]
            print(f"day {day:>3}: {old:>10} -> {new:<10} {why}")
    print("final tier:", f.tier.value)
```

Output:

```
day  97: cloud_only -> hybrid     promote: ip_value 0.648, pressure 0.908
day 201:     hybrid -> client_ok  promote (needs approval): ip_value 0.36 <
0.4
final tier: client_ok
```

## A.4 simulate.py: Section 8 simulation

The full SimPy model (about 120 lines) is in the project files at `paper/code/simulate.py`. Its core is a batching queue that flushes when the window timer fires or the batch reaches 64 items, and a Markov session generator for the prefetch experiment. It writes `results_batching.csv` and `results_prefetch.csv`, the data behind Figures 3 and 4.